

TEN — Coin Marketplace & Mentorship Booking

Simple System Overview

For: Bishal Sir **Goal of this doc:** Explain how the system works, in plain language, with simple pictures.

1. What is this system, in one sentence?

Students spend **coins** to book a **mentorship session**; a **Coordinator** verifies the payment and picks a **Mentor**; the Mentor runs a live call and issues a **certificate** at the end.

2. The 3 people who use it

flowchart LR

```
S["🎓 Student<br/>Books mentorship using coins"]
C["👤 Coordinator<br/>Checks payment, assigns mentor"]
M["👤🏠 Mentor<br/>Takes the live session"]
```

```
S --> C --> M
```

```
classDef student fill:#0f172a,stroke:#38bdf8,stroke-w
classDef coord fill:#1e1b4b,stroke:#818cf8,stroke-wid
```

```
classDef mentor fill:#042f2e,stroke:#2dd4bf,stroke-wi

class S student
class C coord
class M mentor
```

Each has their own dashboard (webpage):

- Student → /student-dashboard.html
- Coordinator → /coordinator-dashboard.html
- Mentor → /mentor-dashboard.html

3. How the pieces fit together (big picture)

Think of it like a restaurant:

- The **dashboards** are the menu/counter — where people interact.
- The **server (backend)** is the kitchen — it does the actual work.
- The **database** is the pantry — it stores everything.
- **Email & Video** are delivery — they send info and host the call.

```
flowchart TB
    D["📱 Dashboards<br/>(what people see & click)"]
    B["⚙️ Backend Server<br/>(does the work)"]
    DB["🗄️ Database<br/>(stores everything)"]
    E["📧 Email"]
    V["👤 Video Call<br/>(Google Meet / Zoom)"]

    D --> B
    B --> DB
    B --> E
    B --> V
```

```
classDef ui fill:#0f172a,stroke:#38bdf8,stroke-width:
classDef server fill:#1e1b4b,stroke:#818cf8,stroke-wi
classDef data fill:#450a0a,stroke:#f87171,stroke-widt
```

```
classDef ext fill:#3f2d1c,stroke:#fbbf24,stroke-width
class D ui
class B server
class DB data
class E,V ext
```

That's it — 4 boxes. Everything in the system is one of these.

4. Step-by-step: what happens when a student books a mentor

Read this like a story, top to bottom.

```
sequenceDiagram
    actor Student
    actor Coordinator
    actor Mentor

    Student->>Student: 1. Spends coins to book a session
    Student->>Student: 2. Uploads payment proof (UTR numb
    Note over Student: System saves this in the database

    Student->>Coordinator: 3. Coordinator gets an instant
    Coordinator->>Coordinator: 4. Checks the payment
    Coordinator->>Mentor: 5. Picks a mentor & time slot

    Note over Student,Mentor: 6. Emails go out automatica

    Mentor->>Mentor: 7. Gets notified of new session
    Mentor->>Student: 8. Starts the live video call
    Mentor->>Student: 9. Session happens

    Mentor->>Student: 10. Marks it complete & grants cert
```

In plain words:

#	What happens
1	Student picks a mentorship and pays using coins
2	Student uploads proof of payment
3	Coordinator instantly sees the new request
4	Coordinator checks the payment is real
5	Coordinator assigns a mentor + time
6	Student & Mentor get automatic emails
7	Mentor sees the new session on their dashboard
8	Mentor clicks one button to start the video call
9	The mentorship session happens
10	Mentor marks it done, student gets a certificate

5. What information the system remembers

Simple version — 6 things get stored:

```
flowchart LR
    ST["👤 Student<br/>name, email, coins"]
    RD["💰 Redemption<br/>the booking + payment"]
    MN["👤🏠 Mentor<br/>name, expertise, free slots"]
    CO["👤 Coordinator<br/>name, which domains they manage"]
    CR["🏆 Certificate<br/>issued after session"]
    AL["📖 Activity Log<br/>a record of everything that h"]

    ST --> RD
    RD --> MN
    RD --> CO
    RD --> CR
    RD --> AL
```

```
classDef box fill:#1e1b4b,stroke:#818cf8,stroke-width
class ST,RD,MN,CO,CR,AL box
```

In short: **a booking (Redemption) connects a Student, a Mentor, and a Coordinator — and it can produce a Certificate.** Everything that happens is written to the Activity Log for record-keeping.

6. What changed on the dashboards, and why

Dashboard	Problem before	What we fixed
Coordinator	Had to scroll through every request, all domains mixed together	Added a filter so they only see requests for their domain
Coordinator	Assigned mentors manually over chat/email — caused double-booking	Added a built-in "assign mentor + slot" button that checks real availability
Mentor	Had to manually create and share a meeting link	Added a "Launch Meeting" button — one click starts the call

```
flowchart LR
    A["Coordinator sees<br/>only their domain"] --> B["Co

classDef step fill:#042f2e,stroke:#2dd4bf,stroke-widt
class A,B,C,D step
```

7. The whole thing in one picture

flowchart TD

```
S["🎓 Student books & pays with coins"] --> R["💻 Sa  
R --> ALERT["🔔 Coordinator gets live alert"]  
ALERT --> CHECK["  Coordinator verifies payment"]  
CHECK --> ASSIGN["  Coordinator assigns a Mentor"]  
ASSIGN --> MAIL["📧 Auto email sent to Student & Ment  
ASSIGN --> NOTIFY["🔔 Mentor gets live alert"]  
NOTIFY --> CALL["📞 Mentor launches video call"]  
CALL --> DONE["★ Certificate issued to Student"]
```

```
classDef student fill:#0f172a,stroke:#38bdf8,stroke-w  
classDef system fill:#450a0a,stroke:#f87171,stroke-wi  
classDef coord fill:#1e1b4b,stroke:#818cf8,stroke-wid  
classDef mentor fill:#042f2e,stroke:#2dd4bf,stroke-wi  
classDef done fill:#052e16,stroke:#4ade80,stroke-widt
```

```
class S student  
class R system  
class ALERT,CHECK,ASSIGN coord  
class MAIL system  
class NOTIFY,CALL mentor  
class DONE done
```

That's the whole system. If you'd like, I can also make the full detailed technical version (with database field names, API routes, and the C4 architecture diagram) for the engineering team — this simple version is meant for a quick walkthrough.